CS 350
From Data Mining to Deep Learning

Instructor:
Dr. Peilong Li

Lecture 11:
Large Language Models

Lecture outline

Announcements/notes

- Project milestone due this Fri
- Prepare your presentation slides (2 weeks)

Today's lecture

- Large Language Models
  - Transformer Models
  - Use case: Topic Modeling

NLP techniques
In timeline

Symbolic NLP (1950s to 1990s)

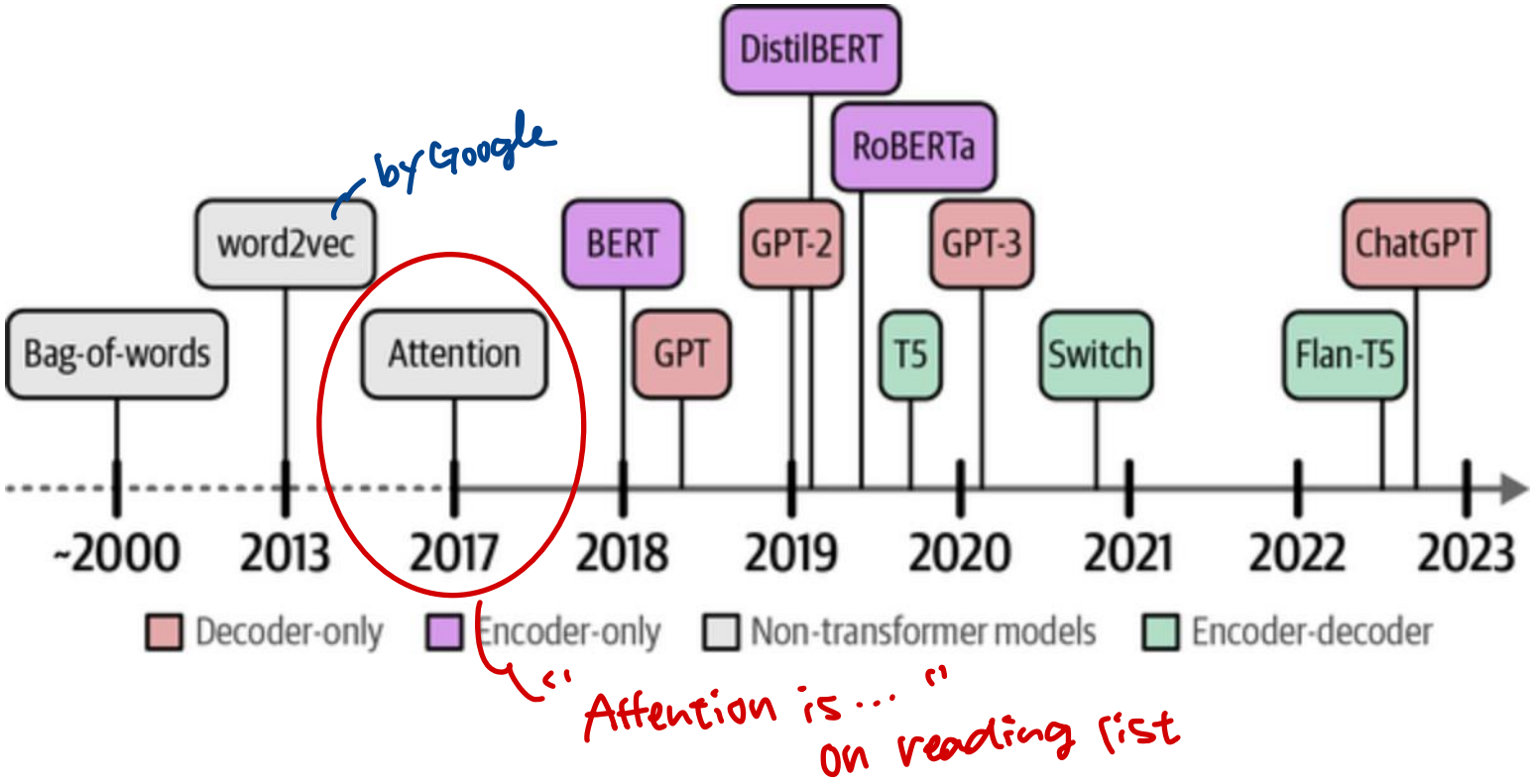
Statistical NLP (1990s to 2010s)

Neural NLP (2010s to present)

DEEP LEARNING FOR NLP

Evolution of Language Models

- Source: Alammar and Grootendorst (2024)



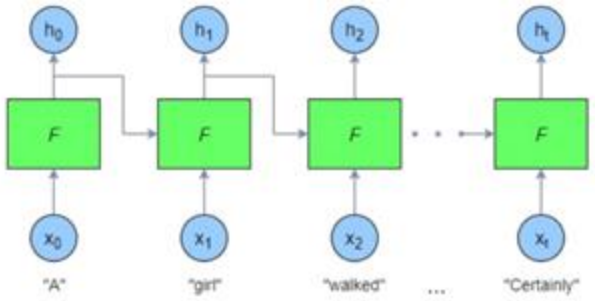
Deep Learning Models

Neural Networks → RNN → Encoder-Decoder

Word to representation (word2vec)

- Layered structure
- True targets vs output predictions
- Weights and loss functions
- Optimizers

Language Generation



Translation

- Joint RNNs
- Creating context and using the final context of first RNN
- Training using parallel corpora

Image source: https://www.slideshare.net/darvind/nlp-using-transformers

Deep Learning Models

Attention Models

Translation and Image Recognition

- Pay selective attention
- Utilize intermediate states while decoding
- Do alignment while translating

Transformer

Translation

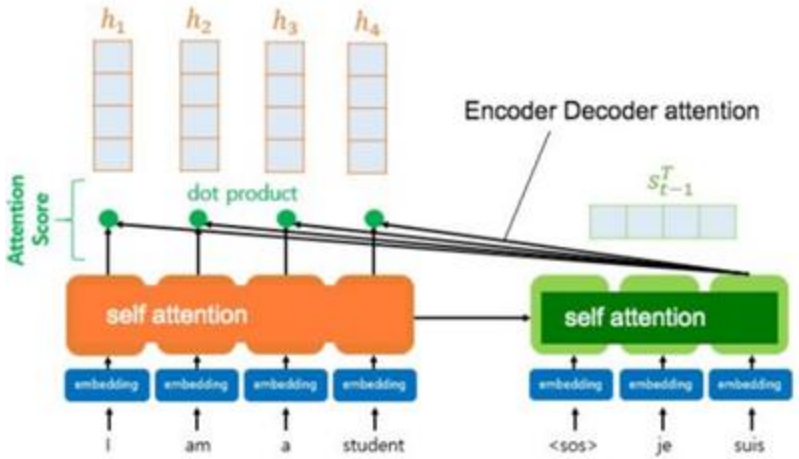


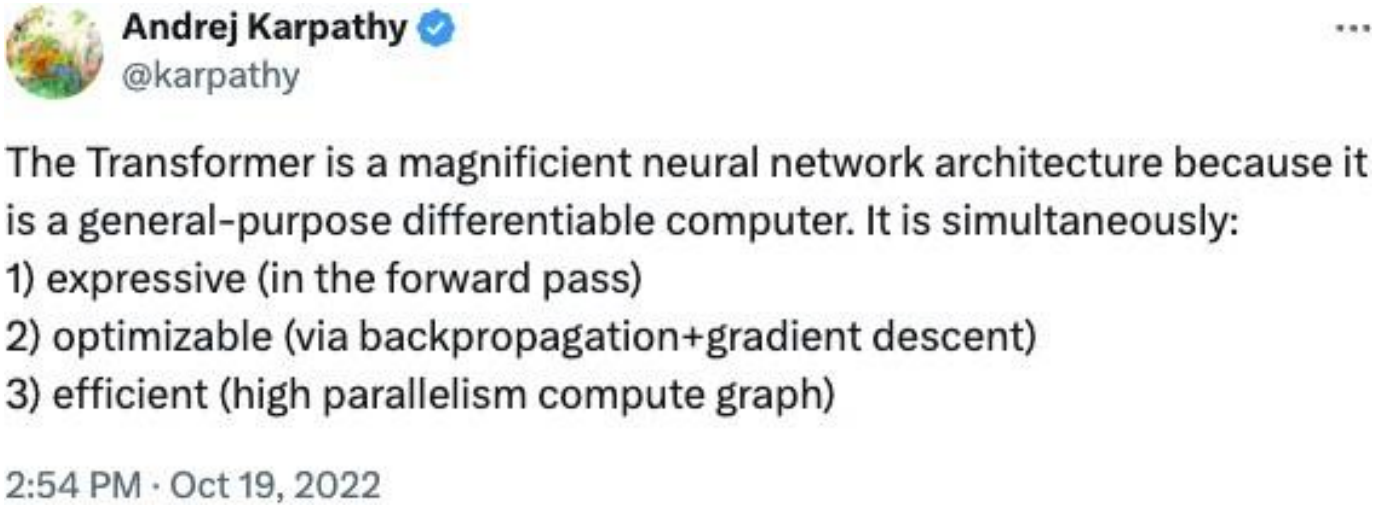
Image source: https://www.slideshare.net/darvind/nlp-using-transformers

Transformer Models

- This architecture aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. It relies entirely on self-attention to compute representations of its input and output WITHOUT using sequence-aligned RNNs or convolution

"Attention is All You Need (2017)"

- By **Google Brain**: Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L. and Gomez, A. N., Kaiser, L. and Polosukhin, I. (NIPS 2017)
- It revolutionized neural networks by introducing the Transformer, a highly flexible architecture enabling LLMs like BERT, GPT, and T5.
- The core innovation of self-attention allows each token to capture global context efficiently, eliminating the need for recurrence.



Transformer Architecture

- Encoder processes input
- Decoder generates output, predicting the next token auto-regressively
- Feed forward network (deep learning)
- Multi-head self-attention
- Masked attention for decoder
- Positional encoding
- Check PyTorch function: Docs>torch.nn>Transformer

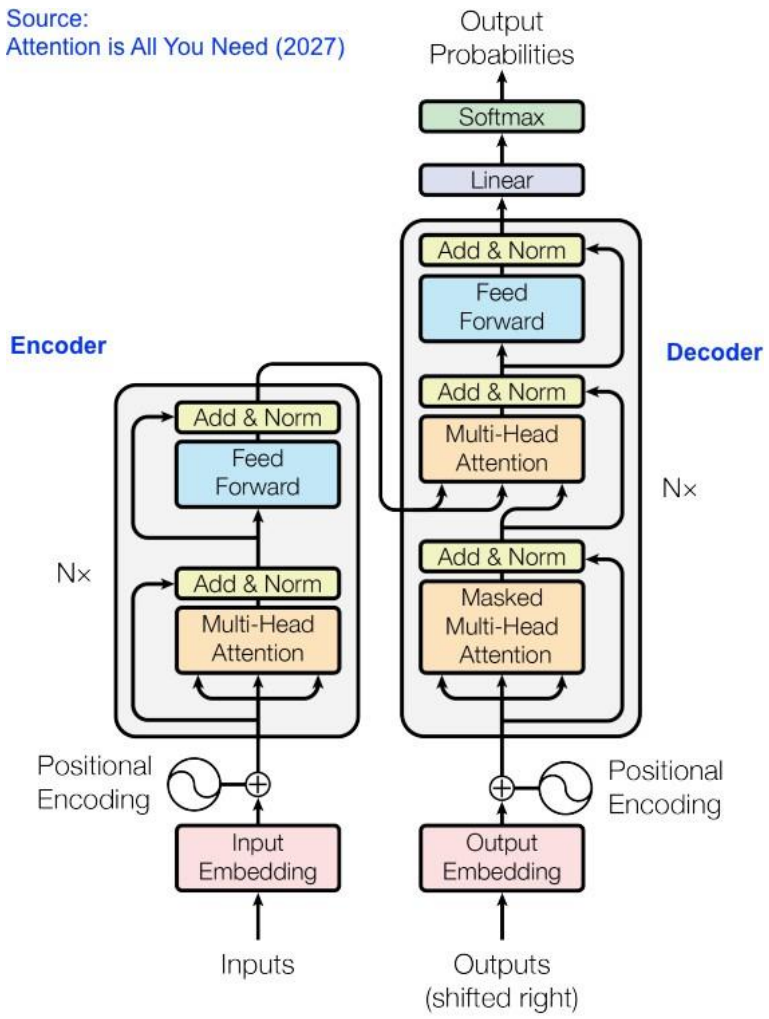


Figure 1: The Transformer - model architecture.

What is Self-Attention

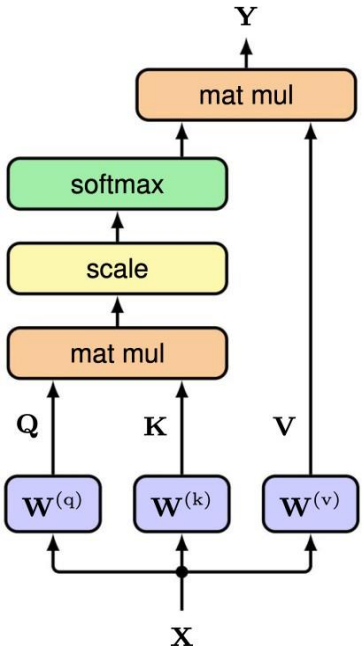
- Self-attention
  - A rich representation which captures the interdependencies between the words compared to single word embeddings
  - Parallel computation
  - Adhere to long-term dependencies

Why Is It Important?

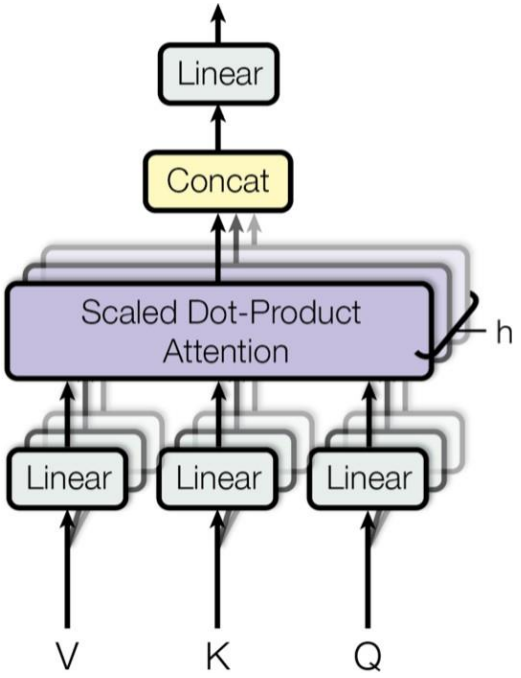
- Analogy
  - Imagine you're reading a long text message, and you want to quickly understand what it's about. Instead of reading word by word, you skim through the whole message, paying attention to certain words like "party," "Saturday," or "invite." These key words help you get the main idea without needing to go through everything slowly.

Attention Mechanism

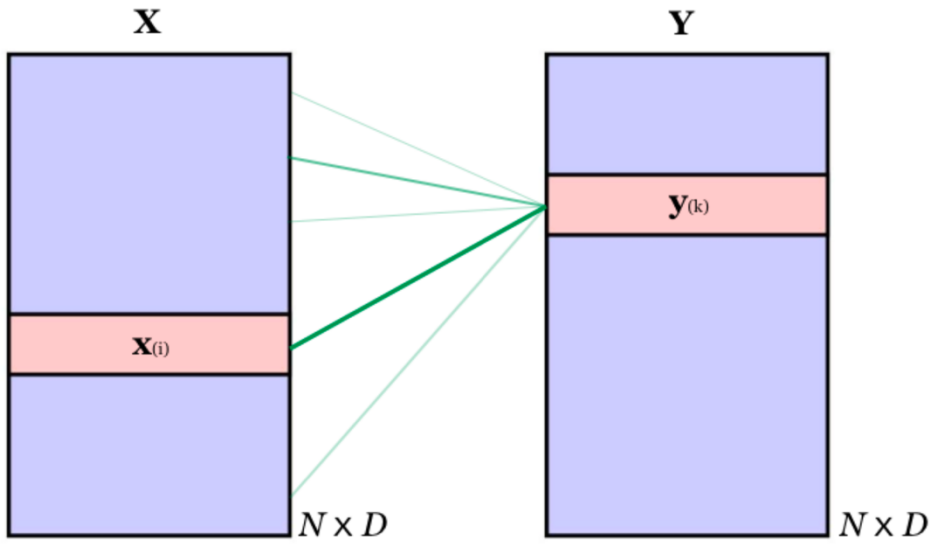
- Single-head attention
  - Think of it as having one spotlight that only illuminates one aspect of the input at a time.



- Multi-head attention
  - Allows the model to focus on different relationships or patterns within the input



Scaled Dot-Product Self-Attention



$$\mathbf{y}_{(k)} \leftarrow \sum_{i=1}^{N} \alpha_{ki} \mathbf{x}_{(i)}$$

$$\alpha_{ki} = \frac{\exp(\mathbf{x}_{(k)} \mathbf{x}_{(i)}^T)}{\sum_{j=1}^{N} \exp(\mathbf{x}_{(k)} \mathbf{x}_{(j)}^T)}$$

Expressed in matrix form:

$$\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$$

(Q,K,V)-parameterization: $\mathbf{W}^{(q)}, \mathbf{W}^{(k)}, \mathbf{W}^{(v)} \in \mathbb{R}^{D \times D}$

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{X}\mathbf{W}^{(q)}(\mathbf{X}\mathbf{W}^{(k)})^T\right]\mathbf{X}\mathbf{W}^{(v)} \equiv \text{Softmax}[\mathbf{Q}\mathbf{K}^T]\mathbf{V}$$

Scaled self-attention: $\mathbf{Y} = \text{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D}}\right]\mathbf{V} \equiv \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$

Attention Calculation by Example

Step 1: Sequence of words

["the", "cat", "sat", "on", "the", "mat"]

where each word is represented by its embedding, e.g.:

"cat" -> [0.56, -0.27, 0.44]

Step 2: Queries, Keys, and Values are generated for each word:

”the" -> Q1, K1, V1

"cat" -> Q2, K2, V2

"sat" -> Q3, K3, V3

For example,

Q1 = [0.5, 0.1, 0.3], Q2 = [0.6, 0.2, 0.8], Q3 = [0.2, 0.5, 0.4]

K1 = [0.4, 0.1, 0.2], K2 = [0.7, 0.3, 0.6], K3 = [0.3, 0.7, 0.2]

V1 = [0.3, 0.2, 0.1], V2 = [0.5, 0.4, 0.2], V3 = [0.4, 0.1, 0.5]

Attention Calculation by Example

Step 3: Focus on "cat" (Q2) and compare with all other words (K1, K2, K3, ...). For example, "cat" (Q2) compares with K3 (from "sat")

```
dot(Q2, K3) = (0.6 * 0.3) + (0.2 * 0.7) + (0.8 * 0.2) = 0.18 + 0.14 + 0.16 = 0.48
```

Step 4: Generate attention weights (focus):

Let's assume after applying softmax, the weights might look like this:

[0.1 (The), 0.5 (cat), 0.9 (sat), 0.1 (on), 0.1 (the), 0.2 (mat)]

(Higher weight for "sat" since it's most relevant to "cat")

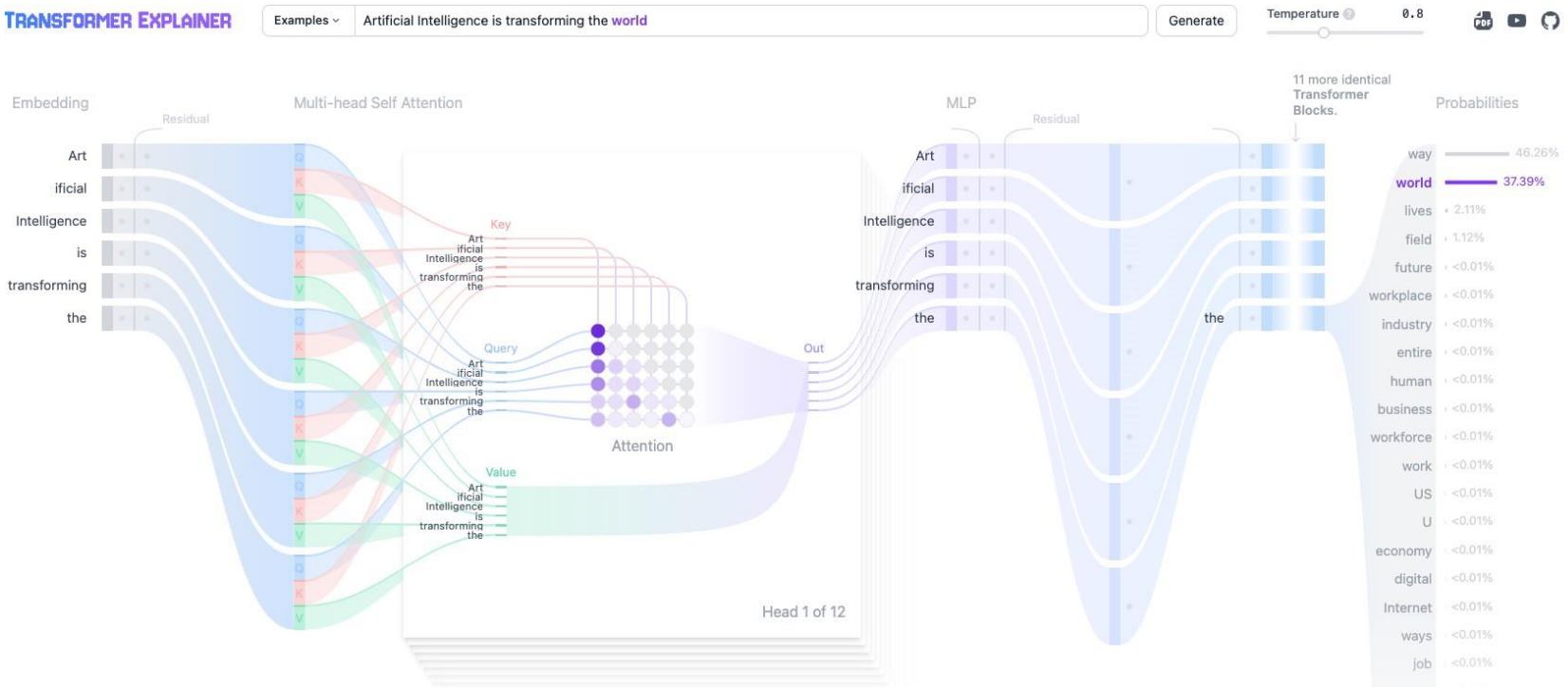
Step 5: Use weights to combine Values (V1, V2, V3, ...):

Output focuses mostly on the meaning of "sat" and "cat"

Output = (0.1 * V1) + (0.5 * V2) + (0.9 * V3) + (0.1 * V4) + ...

Transformer Explainer

- Transformer Explainer: Interactive Learning of GPT-2 by Cho et al.(2024)



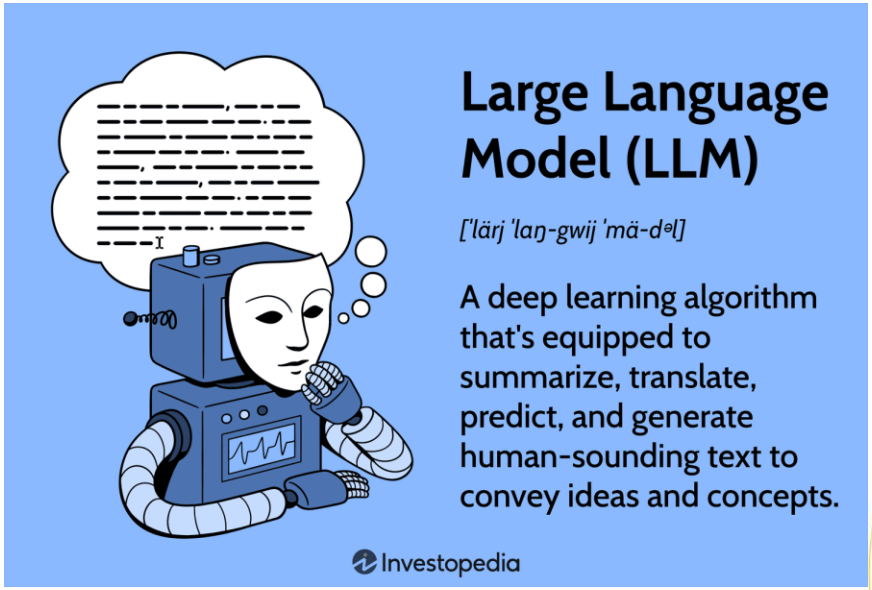
Transformer Challenges ✓

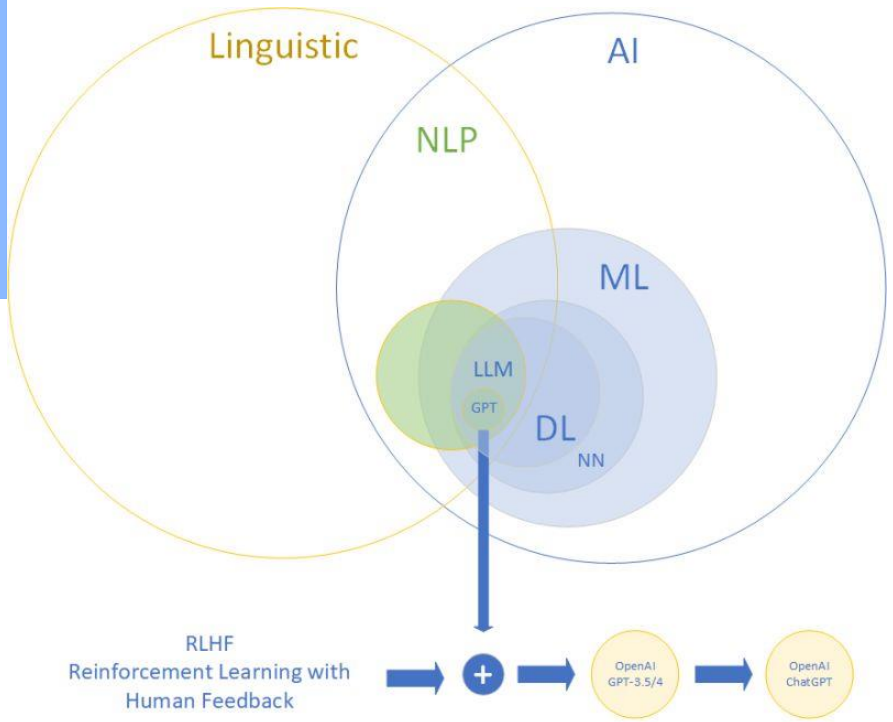
- Attention can only deal with fixed-length text strings. The text has to be split into a certain number of segments or chunks before being fed into the system as input
- This chunking of text causes context fragmentation.

- Solutions?
  - Transformer XL for language modelling
  - Google's BERT is the first attempt

LARGE LANGUAGE MODELS

What are LLMs?



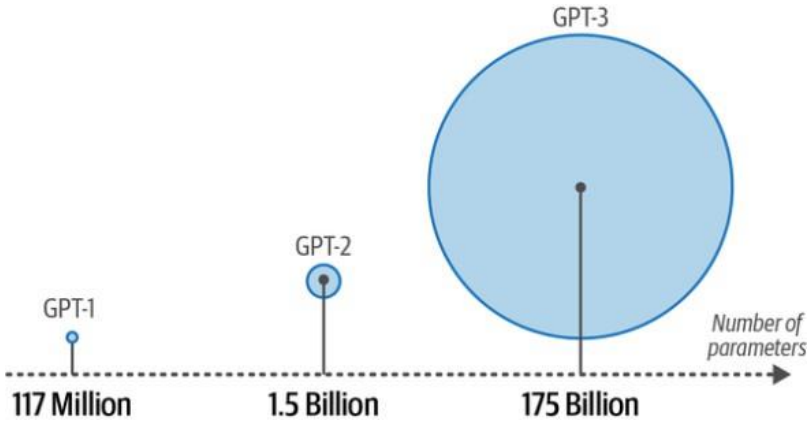


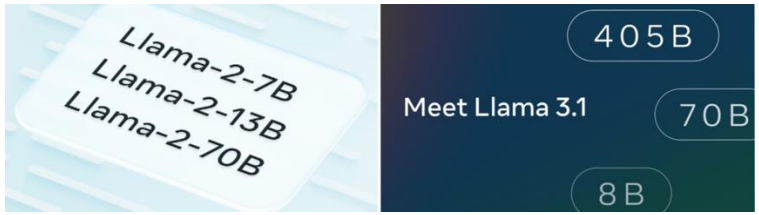
Characteristics of LLMs

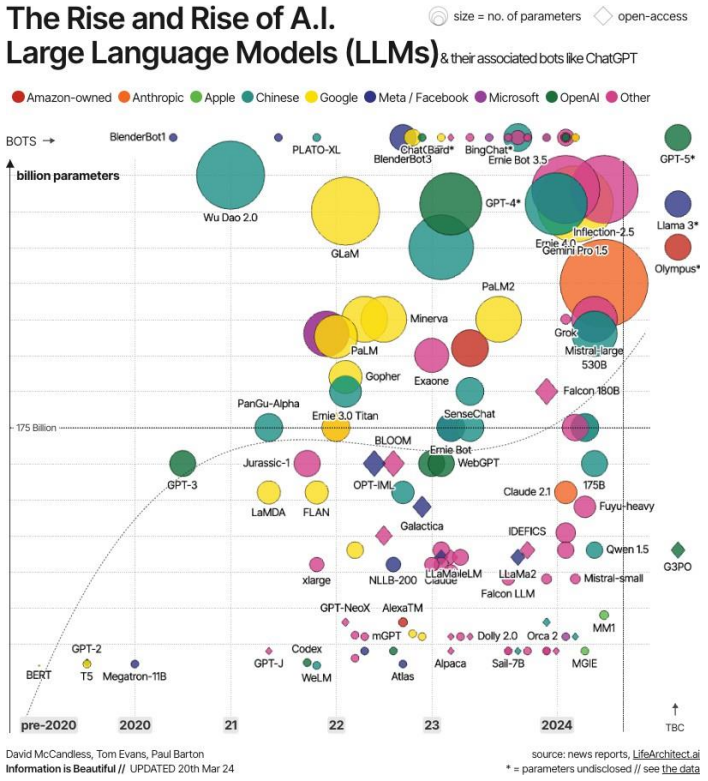
- Large
  - Large training dataset (peta/zeta-byte scale)
  - Large number of parameters (billions scale)
- General Purpose
  - Commonality of human languages
  - Resource restriction
- Pre-trained and Fine-tuned
  - Pre-train a LLM for general purpose with a large dataset
  - Fine-tune it with specific aim with a much smaller dataset

Growing Scales of Language Models

- Scaling laws: increasing parameters, data quality, and compute resources generally improves LLM performance.







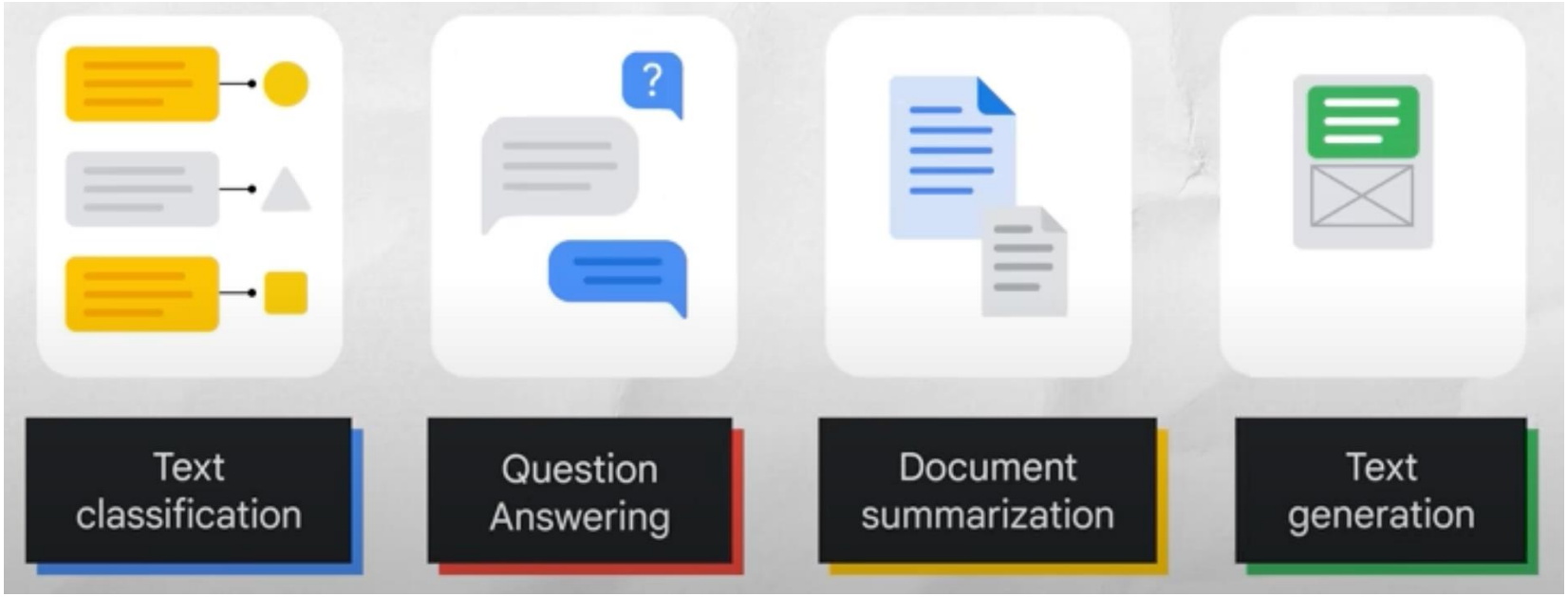
Click into VizSweet

Specialized Transformer LLMs

- **Encoder-Only Models**: BERT (Bidirenctional Encoder Representations from Transformers), DistilBERT, RoBERTa, DeBERTa, etc. Ideal for understanding tasks such as text classification, sentiment analysis, and named entity recognition, with bidirectional attention capturing full context.
- **Decoder-Only Models**: GPT (Generative Pretrained Transformer), GPT-2, GPT-3, and beyond. Optimized for generation tasks, with unidirectional attention. Live example: ChatGPT (GPT-3.5+)
- **Encoder-Decoder Models**: T5 (Text-to-Text Transfer Transformer), BART. Balance input understanding with output generation, suitable for tasks like translation, summarization, and paraphrasing.
- Highlight: Encoder-only models (BERT) are for understanding tasks, while decoder-only models (GPT) are for generative tasks.

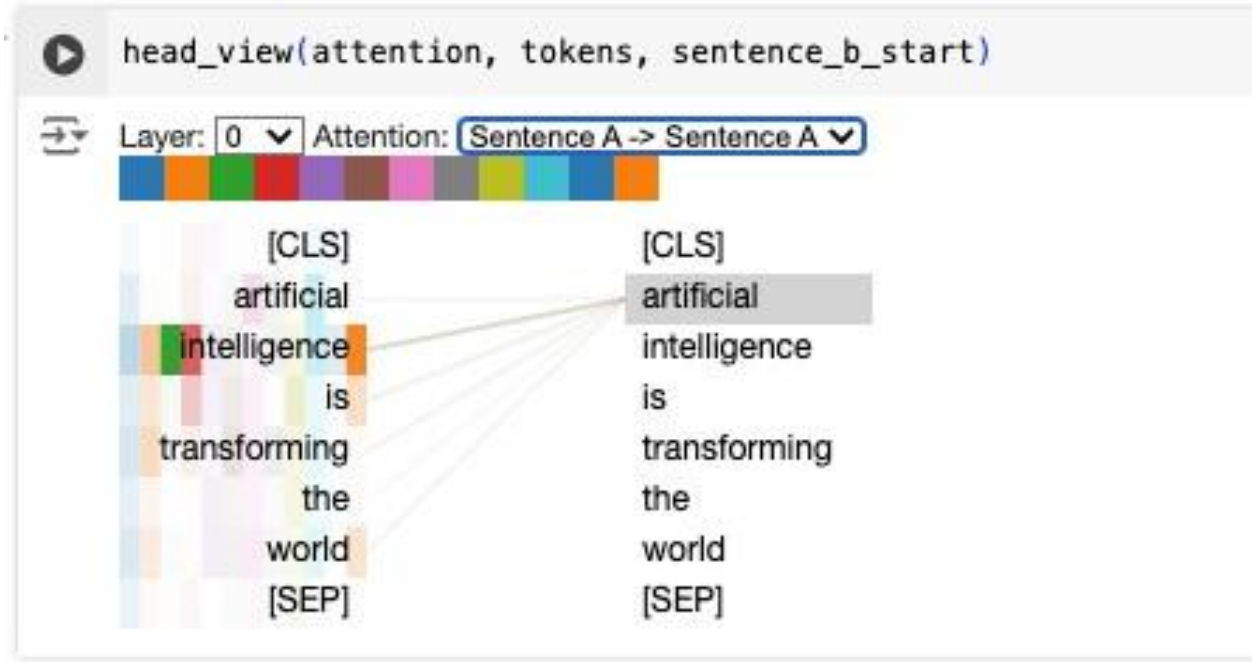
What Can LLMs Do?

- LLMs are trained to solve common language problems, like:



Demo: BertViz

- BertViz: BertViz: Visualize Attention in NLP Models (BERT, GPT2, BART, etc.) https://github.com/jessevig/bertviz



Try it on Google Colab: BertViz Interactive Tutorial

USE CASE: TOPIC MODELING

Topic Modeling

Here's the agenda for our team meeting on Fri.

project

The final cost of the project comes to $1MM.

project $$$

I've attached the receipt of my travel expenses.

$$$

Can you forward that request to my admin?

project

How was your honeymoon?

personal

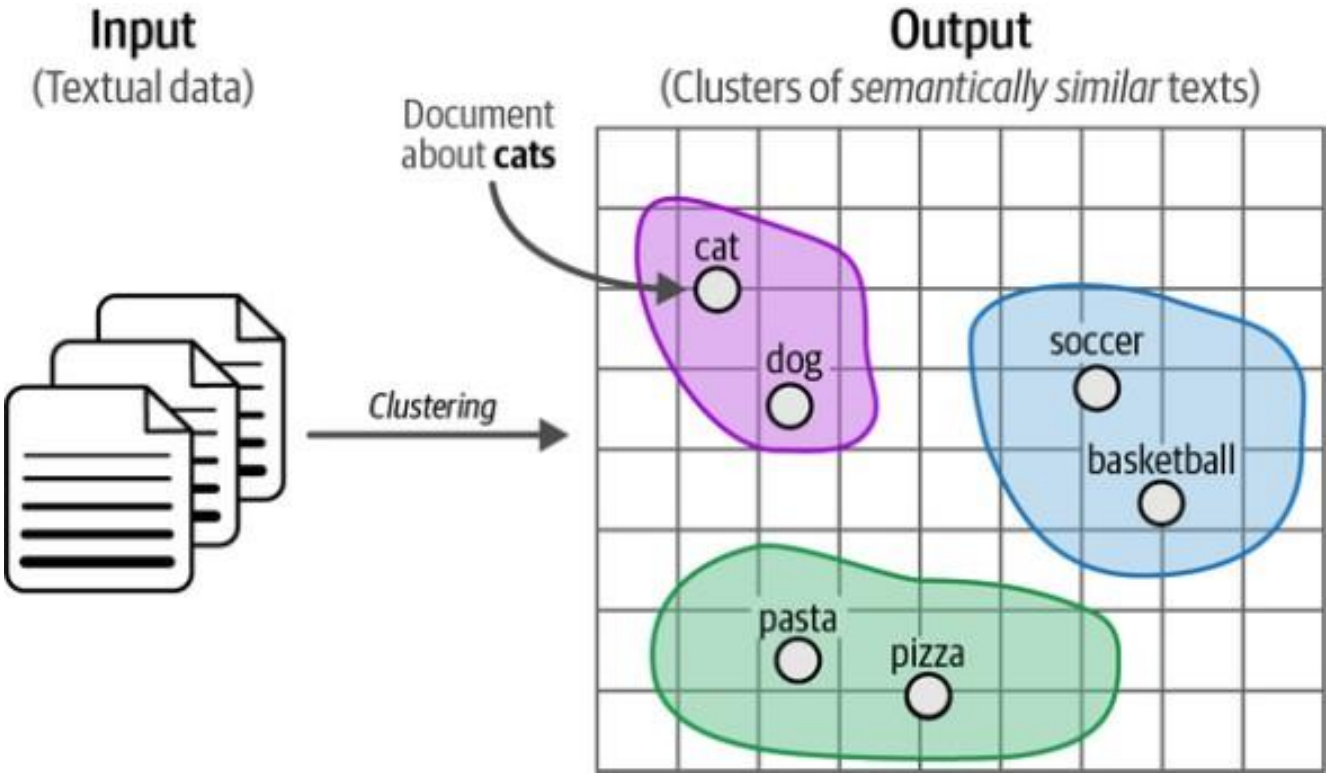
Here are the financial docs you asked for.

$$$

Topic Modeling

Topic Modeling

- Interpreting Context Embeddings



Topic Modeling

- Here's a structured process to interpret contextual embeddings:
  - Dimensionality reduction for effective clustering
  - Extract topics for each cluster
  - Further dimensionality reduction for 2D or 3D visualization

Dimensionality Reduction

Technique	Type	Strength	Weakness	Best Use
PCA	Linear	Captures maximum variance	Assumes linear- ity, less flexible	Medium data with linear relationships
t-SNE	Nonlinear	Preserves local structure	Computationally expensive	Small data, ideal for 2D/3D visualization
UMAP	Nonlinear	Balances local & global structure, scalable	Requires tuning	Clustering in large, complex datasets
Random Projection	Linear (Random)	Fast, scalable, pre- serves distances	Lowerinter- pretability	High-dim data with simple structure
AE (Auto-Encoders)	Nonlinear	Learns nonlinear relationships, cus- tomizable	Requires tuning & significant data	Complex datasets with non-linear pat- terns
VAE (Varia- tional AE)	Nonlinear (Proba- bilistic)	Captures data vari- ability, generates new data	Complex train- ing, requires tuning	When data variabil- ity and generation are important
MDS	Nonlinear	Flexible metrics, preserves distances	Computationally intensive	Semantic similarity in embeddings

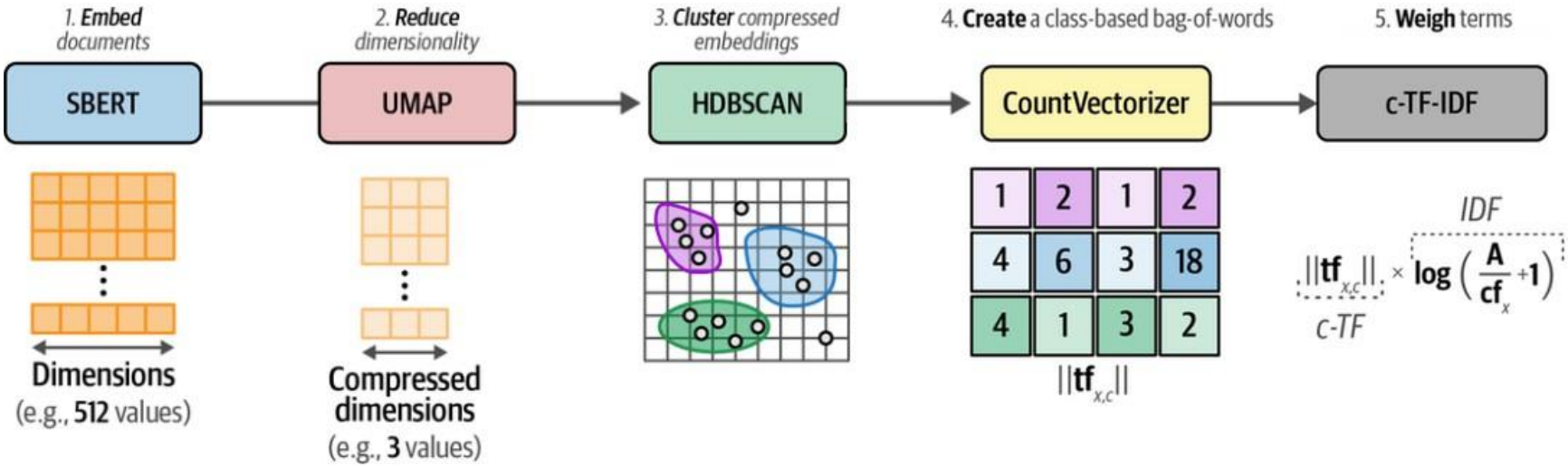
Clustering

Technique	How It Works	Advantages	Limitations	Best Use
k-Means Clustering	Minimizing distances to centroids	Simple, efficient, scalable	Requires k in advance; assumes spherical clusters	When clusters are approximately spherical and equally sized
Agglomerative /Hierarchical Clustering	Merges closest clusters iteratively, forming a hierarchy	Captures hierarchical structure, no need for k	Computationally intensive, sensitive to noise	Data with inherent hierarchical structure
DBSCAN	Groups densely packed points; labels sparse points as outliers	Handles non-convex shapes, detects outliers	Sensitive to parameters, no good for varying densities	Arbitrary shapes, outlier detection, unknown clusters
Spectral Clustering	Uses similarity matrix and eigenvalues for clustering	Good for non-convex clusters, adaptable similarity measures	Computationally expensive, requires k	Small data with complex relationships

Topic Extraction Techniques

Technique	Description	Use in Clusters
TF-IDF (Term Frequency – Inverse Document Frequency)	Identifies important terms by comparing term frequency in a document to its frequency in the corpus. High scores indicate terms important to the document but uncommon in the corpus.	Applied to each cluster to find keywords that best represent its content.
KeyBERT (Keyword Extraction with BERT)	A transformer-based method using BERT embeddings to extract semantically relevant keywords.	Identifies representative keywords or phrases, providing rich, contextually accurate topic descriptions for each cluster.
LDA (Latent Dirichlet Allocation)	A topic modeling technique that assumes documents are mixtures of topics, each with a unique word distribution.	Extracts coherent topics from clusters, revealing specific themes within broader topics.

BERTopic Pipeline for Topic Modeling



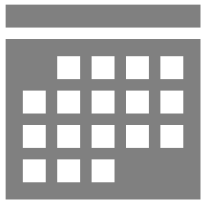
Let's Dive In!

- Find the notebook on Canvas
- Dataset:
  - Rotten Tomato movie reviews

Final notes

Next time:

PE7



Reminders:

Project milestone due this Fri

Prepare your presentation slides (2 weeks)